

Support Triage Bot — Plan de Mejoras

Detalle técnico de los 8 cambios propuestos, listos para aplicar sobre el workflow actual

Sebastián

Agosto 2026

Índice

0. Cómo usar este documento	2
1. Fusionar el modo degradado con el flujo principal	2
Problema	2
Solución	3
2. Centralizar IDs y valores fijos en un nodo “Set”	4
Problema	4
Solución	4
3. Autenticación del Webhook con header secreto	4
Problema	4
Solución	4
4. Respuesta inmediata del Webhook (evitar duplicados)	5
Problema	5
Solución	5
5. Manejo de error al guardar en Google Sheets	6
Problema	6
Solución	6
6. Validación del mensaje entrante	6
Problema	6
Solución	6
7. Link directo a la Google Sheet en el reporte diario	7
Problema	7
Solución	7
8. Reporte semanal agregado	7
Problema	7
Solución	7
9. Orden sugerido de implementación	8

0. Cómo usar este documento

Este documento es el complemento de la documentación técnica general del proyecto (Support_Triage_Bot_Documentacion.pdf). Ahí está el workflow **tal como está hoy**; acá está **cada cambio propuesto**, explicado con el mismo nivel de detalle: qué problema resuelve, cómo se implementa nodo por nodo, y qué conexiones cambian.

Los cambios están ordenados por impacto: primero los que arreglan pérdida de datos o riesgos de seguridad, después las mejoras de calidad, al final las extensiones de producto (reportes semanales).

Resumen ejecutivo:

#	Cambio	Problema que resuelve	Esfuerzo
1	Fusionar el modo degradado con el flujo normal	Tickets con Ollama offline no se guardaban en la sheet	Bajo
2	Centralizar IDs y valores fijos en un nodo "Set"	Mantenimiento disperso, riesgo de inconsistencia	Bajo
3	Autenticación del Webhook con header secreto	Cualquiera con la URL puede inyectar tickets falsos	Bajo
4	Respuesta inmediata del Webhook	Riesgo de duplicados por reintentos de Zapier	Medio
5	Manejo de error al guardar en Sheets	Tickets se pierden en silencio si falla Google Sheets	Bajo
6	Validación del mensaje entrante	Se gasta cómputo clasificando mensajes vacíos	Bajo
7	Link directo a la sheet en el reporte	El reporte diario no permite ir al detalle	Muy bajo
8	Reporte semanal agregado	No hay visibilidad de tendencias	Medio

1. Fusionar el modo degradado con el flujo principal

Problema

Hoy, cuando el nodo **HTTP Request** (la llamada a Ollama) falla —por ejemplo porque Ollama no está corriendo—, el flujo sigue por la segunda salida de error hacia **Send a text message1**, que solo manda un aviso de Telegram ("❌ Ollama offline"). Ahí termina la rama: **el ticket nunca se guarda en la Google Sheet**. Se pierde información real de un cliente.

Solución

En vez de que la salida de error del HTTP Request vaya directo a un Telegram y se corte ahí, la reconectamos hacia el nodo **Validar Respuesta** — que ya tiene, en su bloque catch, toda la lógica necesaria para asignar valores por defecto (categoria: "revision_manual", prioridad: 3, sentimiento: "neutral", revision_manual: true). De esa forma el ticket sigue el mismo camino que cualquier otro: pasa por el **If**, se guarda en **Append row in sheet**, y si además querés mantener el aviso de Telegram de “Ollama offline”, lo agregamos como un nodo adicional en paralelo (no como un final de camino).

Cambio de conexión (antes → después):

Antes:

HTTP Request --(error)--> Send a text message1 [fin del camino]

Después:

HTTP Request --(error)--> Validar Respuesta --> If --> Append row in sheet (+ Telegram si aplica)

HTTP Request --(error)--> Send a text message1 [se mantiene, en paralelo, solo como aviso]

Ajuste necesario en “Validar Respuesta”: como ahora esta rama puede recibir directamente el error del HTTP Request (que no tiene un campo .response con JSON), agregamos una verificación adicional para no romper el JSON.parse:

```
let categoria, prioridad, sentimiento, revision_manual;

try {
  // Si el HTTP Request falló, no va a existir $input.item.json.response
  if (!$input.item.json.response) {
    throw new Error("Sin respuesta de Ollama (posible caída del servicio)");
  }
  const parsed = JSON.parse($input.item.json.response);
  categoria = parsed.categoria || "revision_manual";
  prioridad = Number(parsed.prioridad) || 3;
  sentimiento = parsed.sentimiento || "neutral";
  revision_manual = false;
} catch (error) {
  categoria = "revision_manual";
  prioridad = 3;
  sentimiento = "neutral";
  revision_manual = true;
}

return {
  json: {
    categoria,
    prioridad,
    sentimiento,
    revision_manual
  }
};
```

Resultado: un nodo menos siendo un callejón sin salida, cero tickets perdidos, y el aviso de Telegram sigue funcionando igual que antes.

2. Centralizar IDs y valores fijos en un nodo “Set”

Problema

El ID de la Google Sheet (1Mt0hCwH04kSzLU8CC2oNJT2JjFWHQmvZIUroPFy4kGA) y el chat ID de Telegram (7049611965) están escritos directamente (“hardcodeados”) en 4 nodos distintos (Append row in sheet, Get row(s) in sheet, Send a text message, Send a text message1). Si el día de mañana cambiás de planilla o de chat de Telegram, hay que entrar nodo por nodo a actualizarlo, con el riesgo de olvidarte de alguno.

Solución

Agregar un nodo **Set** (tipo `n8n-nodes-base.set`) justo después del **Webhook**, que defina estos valores una sola vez:

```
{
  "SPREADSHEET_ID": "1Mt0hCwH04kSzLU8CC2oNJT2JjFWHQmvZIUroPFy4kGA",
  "TELEGRAM_CHAT_ID": "7049611965"
}
```

Y en cada nodo que hoy tiene el valor fijo, reemplazarlo por una referencia:

- En **Append row in sheet** y **Get row(s) in sheet**, el `documentId` pasa de ser un valor fijo a `{{ $('Set').item.json.SPREADSHEET_ID }}`.
- En **Send a text message** y **Send a text message1**, el `chatId` pasa a `{{ $('Set').item.json.TELEGRAM_CHAT_ID }}`.

Alternativa más prolija (recomendada a mediano plazo): en vez de un nodo Set, usar **variables de entorno de n8n** (`N8N_SPREADSHEET_ID`, `N8N_TELEGRAM_CHAT_ID`, configuradas en el archivo `.env` o al levantar `n8n start`), accesibles desde cualquier nodo con `{{ $env.N8N_SPREADSHEET_ID }}`. Esto además evita que esos IDs queden visibles si en algún momento compartís el JSON exportado del workflow (como hicimos para armar la documentación).

Resultado: un solo lugar para actualizar, y menos superficie de error humano.

3. Autenticación del Webhook con header secreto

Problema

La URL del Webhook (`http://localhost:5678/webhook/ca9d5f00-c34d-4773-87ab-4c91c2a033ad`) va a quedar expuesta a internet a través de un túnel (ngrok o Cloudflare Tunnel) para que Zapier pueda alcanzarla. Sin ninguna verificación, cualquiera que consiga esa URL (por ejemplo, si queda en un log de ngrok, o alguien la intercepta) puede mandar tickets falsos, saturar tu Telegram, o gastar cuota de Gmail/Sheets.

Solución

Agregar un nodo **If** justo después del **Webhook**, que valide un header secreto antes de dejar pasar el request:

Configuración del nodo “If” de autenticación: - Condición: `{{ $json.headers['x-zapier-secret'] }}` **es igual a** un valor secreto que vos definís (por ejemplo, generado con `openssl rand -hex 16`). - Rama **verdadera** (header correcto): sigue el flujo normal hacia HTTP Request.

- Rama **falsa** (header ausente o incorrecto): corta el flujo. Se puede conectar a un nodo **Respond to Webhook** que devuelva un 401 Unauthorized y listo.

Del lado de Zapier: en el paso “Webhooks by Zapier → POST”, agregar un header adicional:

X-Zapier-Secret: <el mismo valor secreto configurado en n8n>

Dónde guardar el secreto: no en el nodo If directamente (quedaría visible si compartís el workflow), sino como variable de entorno de n8n (N8N_ZAPIER_SECRET), referenciada como `{{ $env.N8N_ZAPIER_SECRET }}` dentro de la condición del If.

Resultado: solo Zapier (que conoce el secreto) puede crear tickets; cualquier otro request queda cortado con un 401 antes de tocar Ollama, Sheets o Telegram.

4. Respuesta inmediata del Webhook (evitar duplicados)

Problema

Por defecto, un Webhook de n8n espera a que **todo** el workflow termine antes de responderle a quien hizo el POST (en este caso, Zapier). Como el HTTP Request a Ollama tiene un timeout configurado de **10 minutos**, si Ollama tarda mucho en responder, Zapier puede interpretar la falta de respuesta como un fallo de su lado y **reintentar el mismo POST** — generando un ticket duplicado en la sheet.

Solución

Cambiar la configuración del nodo **Webhook** para que responda de inmediato, sin esperar el resultado del flujo:

- En los parámetros del nodo Webhook, cambiar "responseMode" de su valor por defecto ("lastNode", espera al último nodo) a **"onReceived"** (responde apenas llega el request, con un simple 200 OK, y deja que el resto del flujo siga procesándose en segundo plano).

```
{
  "httpMethod": "POST",
  "path": "ca9d5f00-c34d-4773-87ab-4c91c2a033ad",
  "responseMode": "onReceived",
  "options": {}
}
```

Con esto, Zapier recibe su 200 OK casi al instante (apenas n8n recibe el POST), y el procesamiento con Ollama, Sheets, Telegram y Gmail sigue corriendo del lado de n8n sin que Zapier esté esperando ni tenga motivo para reintentar.

Complemento opcional (más robusto a futuro): agregar un ID único por mensaje (por ejemplo, un hash simple del texto + la fecha) y, antes de guardar en la sheet, chequear con un nodo de lectura si ya existe un ticket con ese mismo ID — así, incluso si por algún otro motivo llegara un duplicado, no se guarda dos veces.

Resultado: se elimina la ventana de tiempo (hasta 10 minutos) en la que Zapier podía reintentar el envío.

5. Manejo de error al guardar en Google Sheets

Problema

Hoy solo el nodo **HTTP Request** (Ollama) tiene una rama de error configurada. Si el que falla es **Append row in sheet** — por ejemplo, por una cuota de la API de Google Sheets agotada, o un token OAuth2 vencido — el ticket se pierde completamente y en silencio: no queda registro en la sheet, ni aviso de que algo falló.

Solución

Configurar en el nodo **Append row in sheet** las mismas opciones de resiliencia que ya tiene el HTTP Request:

- `retryOnFail: true, maxTries: 2, waitBetweenTries: 2000` (reintenta antes de darse por vencido).
- `onError: "continueErrorOutput"` (si después de los reintentos sigue fallando, no frena todo el workflow, sino que sigue por una segunda salida).

Esa segunda salida se conecta a un nuevo nodo **Telegram** ("Send a text message2"), con un texto de alerta:

```
△ No se pudo guardar el ticket en la sheet
Canal: {{ $('Webhook').item.json.body.canal }}
Mensaje: {{ $('Webhook').item.json.body.mensaje }}
Categoría: {{ $json.categoria }} | Prioridad: {{ $json.prioridad }}
```

Así, aunque el ticket no se llegue a guardar automáticamente, al menos queda un rastro en Telegram con toda la info necesaria para cargarlo a mano.

Resultado: ningún fallo de Google Sheets pasa desapercibido.

6. Validación del mensaje entrante

Problema

Si por algún motivo Zapier manda un POST con el campo mensaje vacío o ausente (por ejemplo, una transcripción de Tactiq que salió en blanco), ese request igual viaja hasta Ollama, gastando tiempo de cómputo del modelo local para clasificar un texto vacío, y termina generando una fila en la sheet sin ningún valor útil.

Solución

Agregar un nodo **If** justo después del Webhook (o inmediatamente después del If de autenticación del punto 3, si se implementa junto), que valide:

- Condición: `{{ $json.body.mensaje }}` **no está vacío** (operation: `notEmpty` o, más estricto, `{{ $json.body.mensaje.trim().length }}` **mayor a 0**).
- Rama **verdadera**: sigue el flujo normal hacia HTTP Request.
- Rama **falsa**: corta el flujo sin llamar a Ollama ni guardar nada (opcionalmente, un aviso de Telegram tipo "❏ Llegó un mensaje vacío desde Zapier — revisar el Zap").

Resultado: se ahorra cómputo innecesario y se evitan filas "basura" en la sheet.

7. Link directo a la Google Sheet en el reporte diario

Problema

El reporte HTML que se manda por Gmail todos los días a las 8:00 tiene las estadísticas del día anterior, pero no ofrece ninguna forma rápida de ir a ver el detalle de cada ticket — hay que ir a buscar la sheet a mano.

Solución

Un agregado mínimo en el nodo **Code in JavaScript1** (el que arma el HTML), justo antes del pie de página:

```
// Agregar antes de la línea "Generado automáticamente por Support Triage Bot"
const linkSheet = `
  <p style="text-align: center; margin-top: 16px;">
    <a
      ↪ href="https://docs.google.com/spreadsheets/d/1Mt0hCwH04kSzLU8CC2oNJT2JjFWHQmvZIUroPFy4kGA/e
        style="color: #0b5d3b; font-size: 13px; text-decoration: none; font-weight:
      ↪ bold;">
        Ver todos los tickets en la planilla →
    </a>
  </p>
`;
```

E insertarlo dentro del template del html (dentro del `const html = \...` ya existente), justo antes del párrafo final “Generado automáticamente...”.

Resultado: un clic desde el mail lleva directo a la sheet completa.

8. Reporte semanal agregado

Problema

El reporte diario es útil para el día a día, pero no permite ver tendencias: ¿el volumen de tickets urgentes está subiendo o bajando semana a semana? ¿qué canal genera más reclamos con el correr de las semanas? Hoy esa información solo se puede sacar mirando la sheet entera a mano.

Solución

Duplicar el Flujo B con una frecuencia distinta:

1. Un segundo **Schedule Trigger**, configurado para dispararse una vez por semana (por ejemplo, los lunes a las 8:00 — en n8n esto se logra con la opción de intervalo semanal, eligiendo el día).
2. Reusar **Get row(s) in sheet** (mismo nodo, o uno igual).
3. Una variante del nodo **Code in JavaScript**, cambiando el rango de fechas de “ayer” a “los últimos 7 días”:

```
const hoy = new Date();
const haceUnaSemana = new Date(hoy);
haceUnaSemana.setDate(hoy.getDate() - 7);
```

```
const items = $input.all();

const ticketsSemana = items.filter(item => {
  const fecha = item.json.Fecha;
  if (!fecha) return false;
  return new Date(fecha) >= haceUnaSemana && new Date(fecha) <= hoy;
});

// El resto de la lógica (total, urgentes, sinClasificar, porCanal, porSentimiento)
// es exactamente la misma que en el reporte diario, aplicada a ticketsSemana
// en vez de ticketsAyer.
```

4. Reusar **Code in JavaScript1** (el armado de HTML), cambiando el título de “Reporte Diario de Soporte” a “Reporte Semanal de Soporte” y el subtítulo de “Resumen del {fecha}” a “Semana del {fecha inicio} al {fecha fin}”.
5. Reusar **Send a message** (Gmail) para el envío.

Resultado: con muy poco código nuevo (reutilizando casi todo lo que ya existe), se obtiene visibilidad de tendencias semana a semana.

9. Orden sugerido de implementación

Para mañana, si querés ir aplicando esto de a poco sin arriesgar el flujo que ya funciona:

1. **Primero, los cambios 1 y 5** (fusionar modo degradado + manejo de error en Sheets) — son los que evitan pérdida real de datos, y no dependen de nada externo.
2. **Después, el cambio 2** (centralizar IDs) — es puramente organizativo, bajo riesgo, y deja el terreno preparado para los cambios de seguridad.
3. **Luego, los cambios 3 y 4** (secreto en el Webhook + respuesta inmediata) — estos sí requieren coordinarlos con un ajuste del lado de Zapier (agregar el header), así que conviene hacerlos juntos y probarlos de punta a punta.
4. **El cambio 6** (validación de mensaje vacío) se puede sumar en cualquier momento, es independiente del resto.
5. **Los cambios 7 y 8** (link en el reporte + reporte semanal) quedan para el final, son mejoras de producto sin urgencia ni riesgo.